Workforce App — Project Documentation

> *WFA-scoped slice of the TalentPop QA App. This document covers only the features the **Workforce Admin (WFA)** role uses day-to-day. For the full app (QA, Forms, Performance Review, etc.), see `PROJECT_DOCUMENTATION.md` .*

1. Project Header

Field	Value
Project Name	Workforce App (WFA-scoped subset of `tp-qa-app`)
Department	Workforce Operations
Engineer(s) Assigned	Abdul Moiz — `abdulmoizrana.ar@gmail.com` / `moiz@talentpop.co`
Stakeholder(s)	Gensis, Leigh
Priority	**P0**
Status	**Live**
Progress %	~85% (active feature work ongoing)
Live URL	https://talentpopapp.com
Staging URL	https://dev.talentpopapp.com
Credentials (dev/staging only)	Email: `moiz@talentpop.co` · Password: `moiz4321` (rotate + move to a password manager — see Section 13)
GitHub Repo (Frontend)	https://github.com/moiz-rana89/qa-ai-app.git
GitHub Repo (Backend)	https://github.com/Talentpop1/qa_ai.git
Last Updated	2026-04-29

2. Project Summary

The **Workforce App** is the WFA team's slice of the TalentPop QA App. It's the operational hub the Workforce Admin team uses to oversee attendance, on-time performance, schedules, and rule-engine-

detected infractions across both **remote agents** and **internal team members**. WFA staff use it to dispute or resolve attendance reports submitted by Team Leads, drill into on-time reporting, manage Hubstaff-synced schedules across the entire org, and approve attendance automation infractions. It exists so WFA can run cross-team audits and arbitration in one place instead of bouncing between Google Sheets, Hubstaff, and ad-hoc TL Slack threads. Outages directly block payroll-adjacent workflows and time-sensitive dispute SLAs, hence the **P0** classification.

---

3. Architecture Overview

The Workforce App is **not a separate codebase** — it's a role-gated subset of the same React SPA. Every WFA-visible page lives in the `tp-qa-app` repo; access is controlled by the role attached to the logged-in user (`wfa` , plus `dev` and `admin` for support/superuser scenarios).

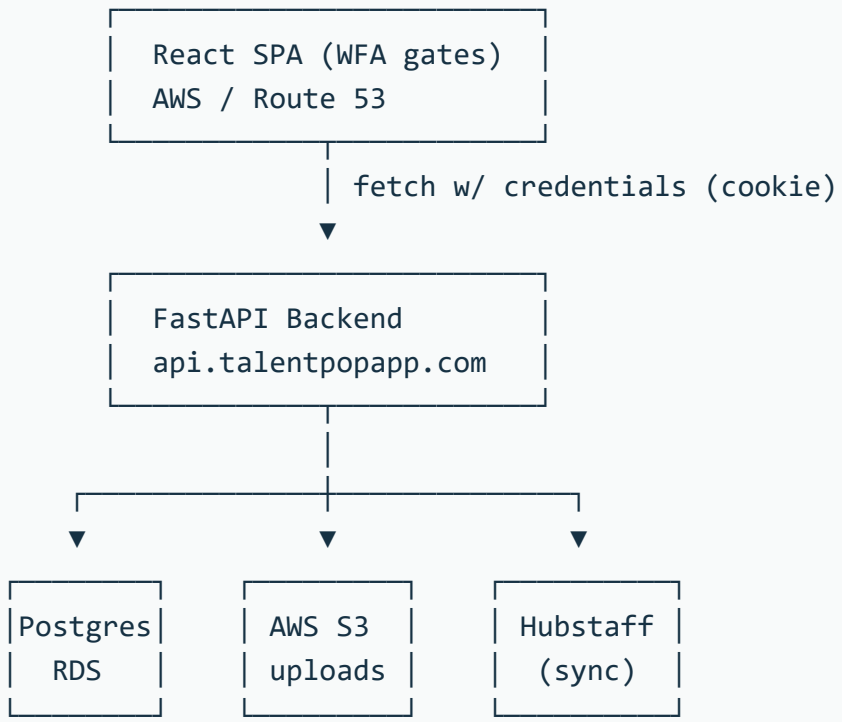
Frontend — React 19 SPA (Vite/rolldown). Tailwind v4 + Ant Design v5 for UI, classic Redux + redux-thunk for state, react-router-dom v7 for routing. Hosted on AWS, DNS via Route 53.

Backend — Shared FastAPI (Python) backend at `https://api.talentpopapp.com` (repo: https://github.com/Talentpop1/qa_ai.git). Cookie-based auth via HttpOnly session cookies; the frontend uses a thin `Api` wrapper that auto-refreshes on 401.

Database — PostgreSQL on AWS RDS (owned by the backend repo).

External Services / APIs —

- **Hubstaff** — schedules and time data are synced via the backend (`/schedules/hubstaff-options` , `mapping_status` , `acknowledge-sync`). This is the most important integration for the WFA role.
- **AWS S3** — file uploads for attendance evidence/attachments via `POST /workforce/reports/upload`
- **Internal automation engine** — the rule-engine that produces attendance infractions (triggered manually via `POST /workforce/reports/run-automation`)


React SPA (WFA gates)
AWS / Route 53
fetch w/ credentials (cookie)
FastAPI Backend
api.talentpopapp.com
Postgres
RDS
AWS S3
uploads
Hubstaff
(sync)

4. Tech Stack

Layer	Tech
Frontend Framework	React 19.1.1 + Vite (rolldown-vite 7.1.14)
Language	JavaScript / JSX (no TypeScript)
State Management	Redux 5 + redux-thunk 3 (classic action-types/reducers pattern)
Routing	react-router-dom 7.9.5
UI Library	Ant Design 5.28 + custom theme
Styling	Tailwind CSS 4.1 + custom CSS overrides
Icons	`@iconify/react` 6
HTTP Client	Native `fetch` via `Api` wrapper (cookie-based)
Notifications	`react-hot-toast` + custom `AntDNotification` wrapper
Date	`dayjs` (newer code) and `moment` (legacy)
Auth	Cookie-based (HttpOnly), `credentials: "include"`, auto-refresh on 401
Backend	FastAPI (Python) — `https://api.talentpopapp.com`
Database	PostgreSQL on AWS RDS
Object Storage	AWS S3 (uploads)
Hosting (Frontend)	AWS (S3 + CloudFront, suspected)
DNS	AWS Route 53
Deployment	Manual `npm run build` + upload of `dist/`
CI/CD	None today
Tests	None today
External APIs	Hubstaff (via backend), AWS S3 (via backend)

5. Key Features (WFA-visible)

WFA Attendance Management

- **WFA Remote Team** (`/wfa-remote-team-attendance`) — Cross-team view of remote agent attendance reports submitted by TLs. WFA can dispute, resolve, or reopen disputes; supports reason picker, notes, multi-file attachments, end-date rules, and "Allow Green Card" approval. Tabs: Resolved by TL, Disputed by WFA, Dispute Resolved by TL.
- **WFA Internal Team** (`/wfa-internal-team-attendance`) — Same flow as remote, scoped to internal team members with department-level filters.
- **WFA Attendance Reporting** (`/wfa-attendance-reporting`) — Read-only cross-team attendance reporting view.
- **WFA OnTime Reporting** (`/wfa-ontime-reporting`) — On-time / late metrics per agent, both remote and internal.
- **Attendance Automation Infractions** (`/attendance-infractions`) — Lists rule-engine-detected infractions; supports live filtering by user/status, default-to-Active, edit drawer with reason / notes / attachments, **Approve (WFA)** action, and **Run Automation** trigger button.

Cross-team operations

- **Schedule Management** (`/schedule-management`) — Hubstaff-synced schedule view across the entire org. WFA users see all schedules (no role-scoped `owner_id` filter — treated identically to admin). Filters: user, client, project, team-lead, schedule type, status, mapping status, date range. Supports schedule creation, edit, mapping update, deactivation, and sync-issue acknowledgement.

Reports / Downloads

- **Download Ticket OR Performance Report** (`/download-report`) — CSV exports filtered by client / agent / event type.
- **Download Client Specific Report** (`/download-client-specific-report`) — Client-form CSV exports.
- **QA AI Report** (`/qa-ai-report`) — CSV downloads of AI-graded QA evaluations.

Auth / Layout

- Cookie-based login at `/login` with auto-refresh on 401.

- All WFA pages render inside `MainLayout` (sidebar + outlet).
- WFA users see the **WFA Attendance Management** section in the sidebar (Remote Team, Internal Team, Attendance Reporting, OnTime Reporting, Attendance Automation Infractions, Schedule Management) plus the **Reports** section.

---

6. API Documentation

All endpoints below are proxied through the `Api` wrapper class with cookie-based auth. Base URL is `${VITE_API_URL}` (production: `https://api.talentpopapp.com`). Every endpoint **requires authentication**.

Auth (`action/auth.js` + `AuthProvider`)

Method	Path	Description	Auth
GET	`/me`	Returns current user from session cookie	Cookie
POST	`/login`	Form-data login (`username` , `password`) → sets cookie + returns `access_token`	Public
POST	`/logout`	Clears the auth cookie	Cookie
POST	`/refresh`	Refreshes session cookie; called automatically on 401	Cookie

Attendance — WFA & TL Views (`action/workforcedashboard.js`)

Method	Path	Description	Auth
GET	`/workforce/reports/attendance-management`	Remote attendance (WFA view)	Cookie
GET	`/workforce/reports/attendance-management-internal-team`	Internal attendance (WFA view)	Cookie
GET	`/workforce/reports/attendance-management-tl`	Remote attendance (TL view, also visible to WFA in some flows)	Cookie
GET	`/workforce/reports/attendance-management-internal-team-tl`	Internal attendance (TL view)	Cookie
GET	`/workforce/reports/attendance-management-resolved`	Resolved attendance (remote)	Cookie
GET	`/workforce/reports/attendance-management-internal-team-resolved`	Resolved attendance (internal)	Cookie
GET	`/workforce/reports/attendance-management-resolved-tl`	Resolved by TL (remote)	Cookie
GET	`/workforce/reports/attendance-management-internal-team-resolved-tl`	Resolved by TL (internal)	Cookie
PATCH	`/workforce/reports/attendance-management/{id}`	Update attendance report (remote)	Cookie
PATCH	`/workforce/reports/attendance-management-internal-team/{id}`	Update attendance report (internal)	Cookie
POST	`/workforce/reports/automation`	Add automation report / advance notice	Cookie

Attendance — Disputes (`action/workforcedashboard.js`)

Method	Path	Description	Auth
POST	`/workforce/reports/attendance/dispute`	Dispute attendance (WFA action)	Cookie
GET	`/workforce/reports/attendance/dispute`	List disputed attendance	Cookie
GET	`/workforce/reports/attendance/dispute/resolved`	List resolved disputes	Cookie
PATCH	`/workforce/reports/attendance/dispute/reopen/{id}`	Reopen a dispute (WFA)	Cookie
PATCH	`/workforce/reports/attendance/dispute/{id}`	Resolve a dispute	Cookie

Attendance — OnTime Reporting

Method	Path	Description	Auth
GET	`/workforce/reports/attendance-ontime`	Remote on-time reports	Cookie
GET	`/workforce/reports/attendance-ontime-internal-team`	Internal on-time reports	Cookie

Attendance Automation Infractions (`action/attendanceInfractions.js`)

Method	Path	Description	Auth
GET	`/workforce/reports/automations/all`	Paginated infraction list with filters	Cookie
GET	`/workforce/reports/automations/{id}`	Single infraction by id	Cookie
PUT	`/workforce/reports/automations/{id}`	Update infraction (reason, notes, attachments, archived, approved-by-wfa)	Cookie
PATCH	`/workforce/reports/automations/{id}/approve-wfa`	WFA approval of an infraction	Cookie
POST	`/workforce/reports/run-automation`	Trigger the infraction automation engine	Cookie

File Uploads

Method	Path	Description	Auth
POST	`/workforce/reports/upload`	Multi-file upload to S3 (used by `UploadFile` component for attendance attachments)	Cookie

Schedule Management (`action/scheduleManagement.js`)

Method	Path	Description	Auth
GET	`/schedules`	List schedules with filters	Cookie
GET	`/schedules/filters`	Available schedule filter values	Cookie
GET	`/schedules/hubstaff-options`	Hubstaff option lists	Cookie
GET	`/schedules/unmapped-count`	Count of unmapped schedules	Cookie
GET	`/schedules/sync-issues-count`	Count of sync issues	Cookie
POST	`/schedules`	Create schedule	Cookie
PUT	`/schedules/{id}`	Update schedule	Cookie
PATCH	`/schedules/{id}/mapping`	Update Hubstaff mapping	Cookie
PATCH	`/schedules/{id}/deactivate`	Deactivate schedule	Cookie
PATCH	`/schedules/{id}/acknowledge-sync`	Acknowledge a sync issue	Cookie
DELETE	`/schedules/{id}`	Delete schedule	Cookie

Filter / dropdown helpers (used across WFA pages)

Method	Path	Description	Auth
GET	`/workforce/reports/get-departments-filter`	Departments	Cookie
GET	`/workforce/reports/get-team-lead-filter`	Team leads	Cookie
GET	`/workforce/reports/get-department-manager-filter`	Department managers	Cookie
GET	`/workforce/reports/get-department-director-filter`	Department directors	Cookie
GET	`/workforce/reports/aom-filter`	AOM list	Cookie
GET	`/workforce/reports/som-filter`	SOM list	Cookie
GET	`/workforce/reports/get_internal_team_member_filter`	Internal-team members	Cookie
GET	`/workforce/reports/get-operations-manager-filter`	OM filter	Cookie
GET	`/workforce/reports/get-associate-operations-manager-filter`	AOM filter	Cookie
GET	`/workforce/reports/get-ops-team-lead-filter`	Ops TL filter	Cookie
GET	`/reports/get_client_names`	Client names	Cookie
GET	`/reports/get_teamlead_names`	Team-lead names	Cookie
GET	`/reports/get_csm_names`	CSM names	Cookie
GET	`/reports/get_om_names`	OM names	Cookie
GET	`/get-team-members-filter`	Team members filter (legacy)	Cookie
GET	`/qa_ai_apis/get-team-members-filter`	Team members filter (QA scope)	Cookie

Reports / Downloads (`action/formsManagement.js`)

Method	Path	Description	Auth
GET	`/openai/client-names`	Client names for download	Cookie
GET	`/openai/agent-names`	Agent names for download	Cookie
GET	`/openai/event-types`	Event types for download	Cookie
GET	`/openai/forms-download`	Ticket / Performance CSV download	Cookie
GET	`/openai/client-form-client-names`	Client names for client-specific download	Cookie
GET	`/openai/client-form-download`	Client-specific CSV download	Cookie
GET	`/qa_ai_apis/qa_ai_report_download`	QA AI CSV download	Cookie

---

7. Database Schema Overview

The DB lives in the backend repo (https://github.com/Talentpop1/qa_ai.git) on **PostgreSQL / AWS RDS**. The tables most relevant to WFA workflows (inferred from API response shapes and field names):

Inferred Table / Model	Key Fields	Notes
`users`	`id`, `name`, `email`, `role`, `owner_id`	`role = "wfa"` for Workforce Admin team
`agents`	`user_id`, `helpdesk_user_id`, `user_name`	Belongs to clients/teams
`clients`	`id`, `client_name`, `hubstaff_client_id`	Maps to Hubstaff client IDs
`attendance_reports`	`id`, `user_id`, `attendance_reason`, `notes`, `attachments`, `start_date`, `end_date`, `status_resolved_tl`, `status_resolved`, `green_card`, `team_lead_id`, `client_id`	Core table — both remote and internal teams
`attendance_disputes`	`id`, `report_id`, `notes_wfa`, `updated_reason_tl`, `file_urls`, status flags	Created when WFA disputes a TL submission
`automations` / `infractions`	`id`, `user_id`, `reason`, `team_lead_note`, `attachment_url`, `start_date`, `end_date`, `archived`, `approved_by_wfa`, `resolved_by_eng`	Powers the Attendance Automation Infractions page
`schedules`	`id`, `user_id`, `client_id`, `team_lead_id`, `project`, `schedule_type`, `status`, `mapping_status`, `startdate`, `enddate`	Hubstaff-synced; mapping_status tracks unmapped/sync-issue states
`attendance_ontime`	`user_id`, on-time/late metrics, `date`	Source for OnTime Reporting

Confirm with backend SQLAlchemy models for the canonical schema.

---

8. Environment Variables

Variable	What it does	Example value
`VITE_API_URL`	Base URL for the FastAPI backend. Used by every `Api.*` call and by the auth `/me`, `/refresh`, `/login` flows.	`https://api.talentpopapp.com` (prod) · `http://localhost:8000` (local)
`VITE_ENCRYPTION_KEY`	Fernet symmetric key for client-side encryption (used by the `fernet` package).	`your_fernet_key_here` (placeholder)

> ***Important:*** *all* `VITE_*` *vars are inlined into the client bundle at build time — they are* ***not secret*** *post-build. Real secrets must live on the backend.*

---

9. Setup Instructions

Prerequisites

- Node.js **20+**
- npm 10+
- Access to a running backend (default: `https://api.talentpopapp.com`)
- A WFA-role test account on dev/staging (see Section 1 — Credentials)

Steps

```
# 1. Clone
git clone https://github.com/moiz-rana89/qa-ai-app.git tp-qa-app
cd tp-qa-app

# 2. Install
npm install

# 3. Configure env
cp .env .env.local
# Set:
#   VITE_API_URL=<backend base url>
#   VITE_ENCRYPTION_KEY=<fernet key>     (ask Abdul Moiz / SecOps)

# 4. Start dev server
npm run dev
# → http://localhost:5173
# Log in with a wfa-role account to access WFA pages

# 5. Lint
npm run lint

# 6. Build for production
npm run build

# 7. Preview production build
npm run preview
```

Tests

No test suite today. Adding Vitest + React Testing Library is on the backlog.

---

10. Deployment Instructions

Hosting: AWS (frontend bundle deployed to AWS — likely S3 + CloudFront; confirm with infra owner).
DNS: Route 53 — `talentpopapp.com` (production) and `dev.talentpopapp.com` (staging). **Trigger: Manual.**

```
# 1. From a clean main branch
git checkout main
git pull

# 2. Install dependencies (if changed)
npm install

# 3. Set the right VITE_API_URL in .env for the target environment
#    - Production:  VITE_API_URL=https://api.talentpopapp.com
#    - Staging:     VITE_API_URL=<staging API URL — confirm>

# 4. Build
npm run build

# 5. Upload dist/ to AWS
#    aws s3 sync dist/ s3://<bucket-for-env> --delete
#    aws cloudfront create-invalidation --distribution-id <dist-id> --
paths "/*"
```

> ***Confirm with infra owner:*** *S3 bucket name, CloudFront distribution ID, who has write access. Add those values here once known.*

Rollback

- Re-build from a previous known-good commit and re-upload `dist/`.
- Or, if S3 versioning is enabled, restore prior object versions.

Backlog

- **Move to GitHub Actions** for automated build + deploy on push to `main` / `staging`.

---

11. Active Tasks / Backlog

In Progress

- Continued polish on the Attendance Automation Infractions edit drawer and live filters.

To Do (WFA-scoped)

- Add automated tests for WFA flows (dispute/resolve, infraction approval, schedule sync acknowledgement).
- Set up CI/CD for build + deploy.
- Migrate `moment` → `dayjs` in any remaining WFA pages.
- Split `AppRouters.jsx` into a `routes/` config + smaller router file.
- Consider a top-level error boundary so a render error in any WFA page doesn't crash the whole shell.

Recently Shipped (WFA-impacting)

- **Attendance Infractions**: live filters (no Apply button), default status = Active, `user_name` column, **Run Automation** trigger button, redesigned edit drawer (sticky Cancel/Save, stacked Archived / Approved-by-WFA checkboxes, multi-file `UploadFile` component with X-to-remove + image/PDF preview).
- **WFA edit drawers** (Remote + Internal): files now persist across reason changes — WFA can keep existing attachments and add/remove individual files via the upload area's X button.
- **Schedule Management**: WFA access added; treated identically to admin (no `owner_id` filter, sees all schedules across the org).
- **OM granted access** to Attendance Infractions page (collaborates with WFA on disputes).

Known Bugs

- Old non-WFA edit drawers (`EditRemoteTeam.jsx` , `EditWorkforceTem.jsx`) still wipe attachment state on reason change — intentionally left untouched but worth revisiting for consistency.

---

12. Known Issues & Limitations

- **No test coverage.** No unit, integration, or E2E tests for any WFA flow today.
- **No CI/CD config in repo.** Lint is the only automated check, and it isn't gated. Builds and deploys are 100% manual.
- **Mixed date libraries.** Older code uses `moment` (deprecated), newer code uses `dayjs`. Migration in backlog.
- `AppRouters.jsx` **is a 565-line god file** with `ROUTE_ROLES` + every route inline. Worth splitting.
- `.env` **is committed** with placeholders / dev URLs. Production deploys must override `VITE_API_URL` and `VITE_ENCRYPTION_KEY` via the build environment.
- **Duplicate filter actions** exist in the action files — multiple variants of team-member filters fetched from different paths. Some are functional duplicates and could be consolidated.
- **No error boundary** — a render error in any page crashes the whole shell.
- **No CSP / SRI** declared in `index.html`.
- **Hubstaff sync** depends on backend job timing — UI surfaces unmapped-count and sync-issues-count but doesn't expose raw sync logs.

---

13. Security Notes

Authentication

- Cookie-based session. Backend issues an HttpOnly cookie on `POST /login`; the frontend never reads it directly. All requests use `credentials: "include"`.
- On 401 from any API call, the `Api` wrapper automatically calls `POST /refresh` (de-duped via a shared `refreshPromise`) and retries once. If refresh fails, the user is redirected to `/login`.
- `AuthProvider` calls `GET /me` on mount and falls back to refresh-then-retry on 401.

Authorization (frontend-only enforcement)

- `ROUTE_ROLES` in `src/layout/AppRouters.jsx` maps each route to a list of allowed roles. WFA-relevant entries:
  - `wfa-remote-team-attendance` → `["dev", "wfa", "admin"]`

  - `wfa-internal-team-attendance` → `["dev", "wfa", "admin"]`
  - `wfa-attendance-reporting` → `["dev", "wfa", "admin"]`
  - `wfa-ontime-reporting` → `["dev", "wfa", "admin"]`
  - `attendance-infractions` → `["admin", "dev", "wfa"]`
  - `schedule-management` → `["dev", "admin", "om", "aom", "tl", "csm", "wfa"]`
  - `download-report`, `download-client-specific-report`, `qa-ai-report` → all include `wfa`

- `ProtectedRoute` checks the current user's role against the allowed list and redirects unauthorized users.
- **The backend (FastAPI) must independently enforce authorization on every endpoint.** Frontend role gating is for UX only, not security. SecOps should verify role checks on every backend route, especially the WFA-only mutation paths (`/workforce/reports/attendance/dispute*`, `/workforce/reports/automations/{id}/approve-wfa`, `/schedules/*` mutations).

Secrets Management

- `VITE_API_URL` and `VITE_ENCRYPTION_KEY` come from `.env` at build time. The repo contains only placeholders/dev URLs.
- All `VITE_*` env vars are **inlined into the client bundle at build time** — they are public post-build. Never put a true secret here.
- Real API keys live on the backend.
- **Dev/staging credentials in this doc** (`moiz@talentpop.co` / `moiz4321`) should be moved to a password vault and replaced here with a reference like "*see Workforce App vault entry*". Current password is also weak — rotate to a strong one.

Active Security Findings (action items)

1. **Weak dev/staging password** (`moiz4321`) — rotate to a strong unique password and store in the team password manager.
2. **No CSP** on `index.html`. Add a strict Content-Security-Policy header at minimum allowing only `self`, the API origin, and (if still embedded anywhere) Google Forms.

CORS

- Backend (FastAPI) must allow `credentials: true` and explicit allow-list of `https://talentpopapp.com` and `https://dev.talentpopapp.com` (no `*` with credentials). Verify in the `qa_ai` repo.

Rate Limiting

- Not implemented at the frontend. Should be enforced at the backend / API gateway (FastAPI middleware or AWS WAF). The **Run Automation** button in particular dispatches a `POST /workforce/reports/run-automation` that triggers a heavy job — confirm backend has rate-limiting / debouncing on this route.

Input Validation

- Frontend does presence and length validation in some places (e.g. 70-char minimum on coaching/notes). The backend must independently validate every payload — never trust the client.
- File uploads go to `POST /workforce/reports/upload` (S3-backed). Backend should: (a) enforce auth, (b) enforce file-size and MIME-type limits, (c) generate the S3 key (don't trust client-supplied filenames), (d) ideally scan for malware. Verify in the `qa_ai` repo.

---

14. Contact

Role	Person
Built by	Abdul Moiz
Assigned engineer	Abdul Moiz — `abdulmoizrana.ar@gmail.com` / `moiz@talentpop.co`
Stakeholders	Gensis · Leigh
SecOps escalation	TalentPop SecOps team